CSE 150: Introduction to Computer Networks

Chris Parsa

Chapter 2 part 1

1

Current Assignments

- Reading:
  - Chapter 2
    - See **Canvas:** "Reading Assignments"!
      - 2.1,2.2,2.4
    - See **Canvas:** "Suggested Problems"!
      - R problems: R3, R5, R8, R11, R12, R13, R19
  - Chapter 1: finished!
    - Make sure to check Canvas: **Practice, practice, practice** module
    - Extra homework problems in the "Ch 1 part2" lecture slides
- Lab2 due Saturday
- Group Assignment 2 due 10/13 at 5pm

2

2

Important for Lab 3

- Complete reading of Section 2.2 BEFORE you go to lab next week!!

3

3

Upcoming Events

- Tutorials with student tutors
  - Schedule on Canvas

4

4

Chapter 2 - Application Layer

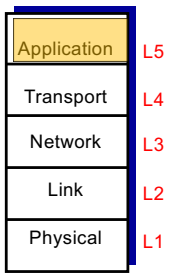
2.1 Principles of network applications

2.2 Web and HTTP

2.4 DNS – fundamental protocol

2.5 P2P applications

2.7 socket programming with UDP and TCP (later for project)



12

Application layer: Overview

Our goals:

- Conceptual *and* implementation aspects of application-layer protocols
  - transport-layer service models
  - client-server paradigm
  - peer-to-peer paradigm
- Focus on 2 essential application-layer protocols
  - HTTP
  - DNS
- Programming network applications
  - socket API

13

Some popular network apps

- social networking
- Web
- text messaging
- e-mail
- multi-user network games
- streaming stored video (YouTube, Hulu, Netflix)
- P2P file sharing
- voice over IP (e.g., Skype)
- real-time video conferencing
- Internet search
- remote login
- …

Q: your favorites?

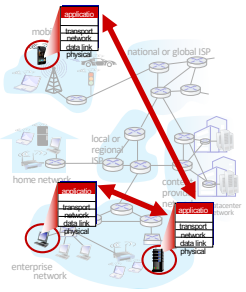
Creating a network app

Write programs that:

- Run on (different) end systems
- Communicate over network
  - e.g., web server software communicates with browser software

No need to write software for network-core devices (routers)

- Network core devices do not run user applications
- Applications on end systems allows for rapid app development, propagation



Network Application architecture models

- Client-server ✓
- Peer-to-peer (P2P)
- Hybrid of client-server and P2P

Client-server paradigm

Architecture: How the application is structured over the various end systems.

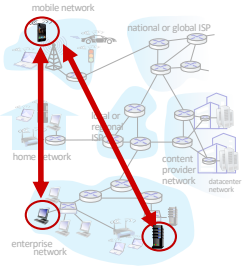
Server:

- always-on host
- permanent IP address
- often in data centers, for scalability and robustness
- Trust

Clients:

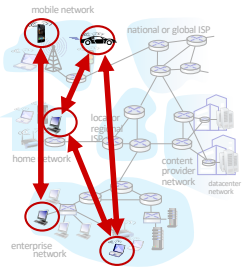
- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- No direct communication with each other
  - Through Server

Example Supporting Protocols: HTTP, DNS and others



Peer-to-peer architecture – "ad hoc"

- *no* always-on server
- Arbitrary end systems communicate directly
- peers receive and provide service to and from their peers
  - P2P file sharing
- *Self scalability* – new peers bring new service capacity, as well as new service demands
- What's a Peer?
  - Any intermittently connected End System
  - Can change IP addresses
  - Can request service from peers and provide service in return (or not!)
  - Peers can be a Client or a Server or both!
- Complex management



A Brief History of P2P Content Distribution, in 10 Major Steps:
https://medium.com/paratii/a-brief-history-of-p2p-content-distribution-in-10-major-steps-6d6733d25122

Hybrid: Client-Server and P2P

Skype/WhatsApp: VOIP --- P2P hybrid App
Instant messaging: Chatting between 2 users

Centralized server (C-S)

- User registers its IP address when it comes online
- *e.g.,* Which friends are currently online?
  - User contacts central server to find IP addresses of buddies and P2P takes over
- Forwarding messages

Client-client connection (P2P)

- Direct (not through server)

lab3 prep (begin)

20

An application-layer protocol defines:

- Types of messages exchanged,
  - e.g., request, response
- Message syntax:
  - what fields in messages & how fields are delineated
- Message semantics
  - meaning of information in fields
- Rules for when and how processes send & respond to messages

Open protocols:

- defined in RFCs, everyone has access to protocol definition
- allows for interoperability
- e.g., HTTP

Proprietary protocols:

- e.g., Skype

21

Section 2.2: Web and HTTP

They are not the same thing!

22

What is the Web

- A distributed repository of information/documents that can be located by URL (Universal Resource Locator) or by clicking on a hyperlink
- Usage:
  Browser Application (Internet Explorer, Firefox, Chrome, Safari etc.) displays web pages that have been requested and received from Web Server
  - HTML defines display format

23

Components

- Client = Browser
- Web Server
- Documents stored on web servers
- HTTP protocol

24

Fun Facts about the Web

- Tim Berners-Lee published 1st website August 6, 1991 while at CERN, in Switzerland. info.cern.ch
  - Today, over 1.7 billion websites registered
- Tim Berners-Lee introduced:
  - the first web server
  - the first browser and editor
  - HTTP (Hypertext Transfer Protocol) in October 1990
  - first version of the HTML (HyperText Markup Language)
- Check out:

http://www.internetlivestats.com/total-number-of-websites/

Timothy Berners Lee

25

What is HTTP?

HyperText Transfer Protocol

- *Lingua franca* of the Web
  - Follow the RFC for HTTP to write an App, you can talk to any web server!!
- HTTP protocol followed to send messages between browser and web server
- HTTP defines the messages transferred, format and expected action upon receipt

26

Message Flow in HTTP

- Client/Server model
- Request ← → Response
- Using HTTP:
  - *Client:* browser requests, receives, and "displays" Web objects
  - *Server:* Web server responds to client requests by sending a response message
- Hardware
- Software

27

Basic Web Page

- Basic web pages start with a *base HTML-file* that can include *referenced objects*
- *Each object* can be stored on a different Web server
  - Web objects: HTML file, JPEG image, Java applet, audio file,…
- Base HTML page and web objects are addressable by a *URL:*

`www.someschool.edu/someDept/pic.gif`

host name — path name

28

HTTP: Client Request message

GET method: request an object from a server

- Two types of HTTP messages: *Request, Response*
  - ASCII (human-readable format) → plaintext
- GET method: request an object from a server

carriage return character
line-feed character

Request Line (methods, URL, version) →

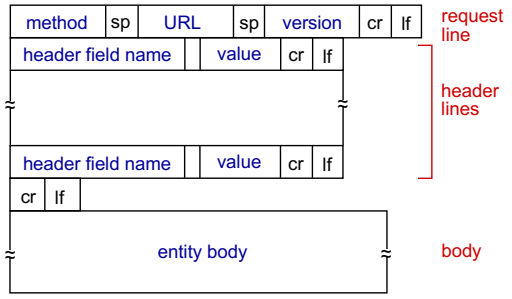
Header Lines

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

End of message: after headers on a new line → carriage return, line feed

29

HTTP Request message: general format



What goes in entity body?

30

Some HTTP request methods

POST
- web page often includes form input
- Send form data from user input → in entity body of message

GET
- Object request OR send form data to server
- Expanded URL: User data included in URL field of the message (following a '?'):

`www.somesite.com/animalsearch?monkeys&banana`

HEAD
- requests headers (only) that would be returned by HTTP GET

PUT
- uploads web object to server
- Uploads object at specified URL with content in body of message

32

HTTP Headers

- Extra information to transfer in the message
- Hundreds – look them up in RFC or Wikipedia

33

HTTP: Server Response message

status line (protocol status code status phrase)

header lines

data, e.g., requested HTML file

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20
  GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007
  17:00:02 GMT\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-
  8859-1\r\n
\r\n
data data data data data ...
```


35

HTTP Response: status codes and phrases

- Status code appears in 1st line in server-to-client response message
- Some sample codes:

  200 OK
  - request succeeded, requested object later in this message

  301 Moved Permanently
  - requested object moved, new location specified later in this message (in Location: field)

  400 Bad Request
  - request msg not understood by server

  404 Not Found
  - requested document not found on this server

  505 HTTP Version Not Supported

36

Using a cache

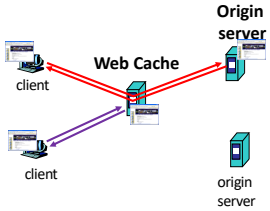
Server Response includes a timestamp timestamp header for the object:

Last-Modified: Mon 22 Jun 1998

Cache stores this date.

Operation:

- Client requests object
  - Requests directed through cache
- If cache has up-to-date version of requested object:
  - Cache returns object directly to Client (?)
- Otherwise, cache:
  - Requests object from Origin Server
  - Stores the object
  - Returns object to Client



Think about Client and Server roles

37

HTTP GET: Client request message

Conditional GET: request updated object from a server

request line (methods, URL, version)

header lines

carriage return character

line-feed character

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
If-Modified-Since: Wed 18 October 2023
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

38

Using a cache -- Conditional GET: Object not Modified

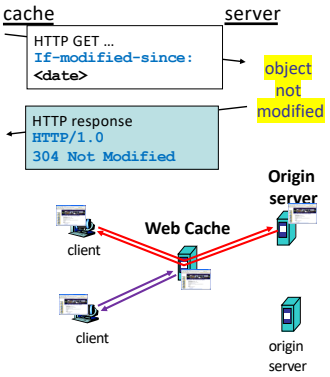
Cache:

Sends Conditional GET to origin server

Server:

If requested object is up-to-date, server does not return the file

Cache forwards response to Client



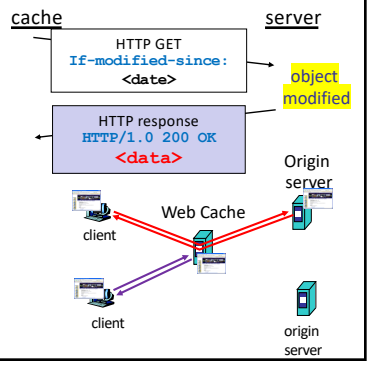
42

Using a cache -- Conditional GET: Object is Modified

Cache:
Uses Conditional GET to origin server

Server:
If object has been updated, server returns the file

Cache:
- Updates its copy of object
- Returns object to Client



43

How do application programs communicate?

45

When operating systems meet networking…

What is a *process*?

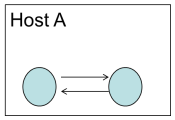
- An actively running program on end system
- OS handles many processes running on computer
  - CPU shares its power with all the processes

46

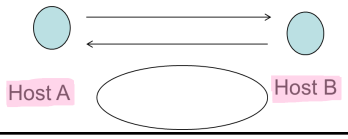
How do Processes Communicate?

Two cases:

1. Same Host: two processes communicate using **Inter-Process Communication (IPC)**
   (defined by OS)
   - How done?



2. Different hosts: processes communicate by **exchanging messages over the network**
   - How?



47

Processes exchange messages

Client process: initiates communication

Server process: waits to be contacted

48

How to Find Server Process on a **remote** host

- Need an address
- Need a port number

49

49

What's an Address?

- An identifier
  - Unique ID (goal globally unique)
  - Shared infrastructure
  - identify user within network
- Types of addresses
  - Flat addresses: student ID number
  - Hierarchical addresses

50

Addressing processes in a network

- To receive messages, process must have *identifier*
- Each Host has a unique 32-bit IP address
- Q: Is this address enough to identify a process running on a host?
  - A: *Many* processes can be running on same host at the same time!

How can we tell them apart?

- *Identifier* includes both IP address and a port number
- Transport Layer: port numbers in header

To send HTTP message to www.gaia.cs.umass.edu web server:
- IP address: 128.119.245.12
- port number: 80

51

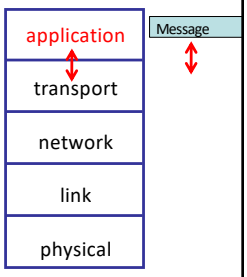
Ports

- Port number is 16 bit number in Transport Layer header
  - Source and Destination
- Servers often "listen" on "well-known" port numbers:
  - HTTP server: 80
  - HTTPS server: 443
  - mail server: 25
- Port number space is organized!
  - 0-1023: Well-known ports
  - 1024 to 49151 : Registered ports
  - 49152–65535: Dynamic/Ephemeral ports
  - Check Wikipedia for complete list
- Operating System assigns Client processes an available port number

52

App Implementation

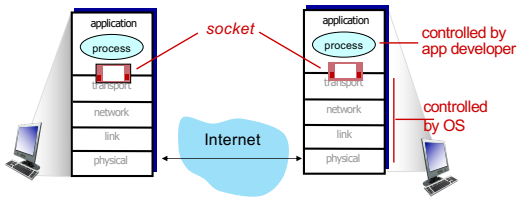
- Application layer uses services of transport layer and network to send and deliver messages
  - API is the hook
  - Send/receive
  - Doesn't need to understand OS!
- Transport Protocols implemented within the OS
  - Windows, Linux, etc. all implement "protocol stacks"
- Entire stack running in End Systems



53

Socket Intetface (API)

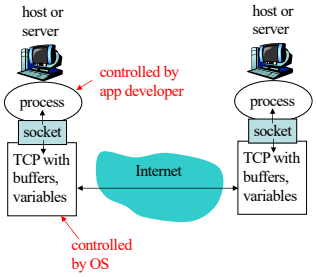
- Sockets allow processes to send and receive messages over the network
- Analogous to door
  - Two sockets: one on each side
  - Sending process shoves message out door into network
  - Reading process accepts incoming message



54

Socket: closer look

- Socket:
  - Endpoint of communication link between 2 processes
  - Processes read and write from/to a socket
    - Similar to writing to or reading from a file: what's different?
  - O.S. allocates resources to each socket
- API – open socket passing (IP Address, destination port)
- options! UDP and TCP sockets



55

What Services does an app need from Transport Layer?

application
transport
network
link
physical

56

What Transport service do Apps need?

Data integrity

- Some Apps require 100% reliable data transfer
  - e.g., file transfer, web transactions
  - **Loss intolerant**
- Others can tolerate some loss
  - e.g., audio, video
  - **Loss tolerant**

Timing

- Some apps require low delay to be "effective", function properly
  - e.g., Internet telephony, interactive games, real-time
  - **Delay sensitive / Delay Insensitive**

Throughput

- Some Apps require a minimum amount to be "effective"
  - (e.g., multimedia)
  - **Bandwidth sensitive**
- Others make use of whatever throughput they get
  - "**elastic apps**"

Security

- encryption, data integrity …

57

What Services are available in the Internet?

TCP service:

- reliable transport between sending and receiving process
- connection-oriented: setup required between client and server processes
- flow control: sender won't overwhelm receiver
- congestion control: throttle sender when detect network is overloaded
- does not provide: timing, minimum throughput guarantee, security

UDP service:

- unreliable data transfer between sending and receiving process
- does not provide: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup,

Q: Why bother? Why is there a UDP?

58

Transport service requirements: common apps

application	data loss	throughput	time sensitive
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5kbps-1Mbps video:10kbps-5Mbps	yes, 100's msec
stored audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	few kbps up	yes, 100's msec
text messaging	no loss	elastic	yes and no

59

Types of Applications

- Delay tolerant
- Delay intolerant/sensitive
- Loss tolerant
- Loss intolerant
- Bandwidth Sensitive
- Elastic

Quiz yourself, practice with terms and find applications for each!

60

HTTP uses TCP because…

Reliable delivery service needed for file transfer

How do we move messages using TCP?

62

Application Layer Message Transfer

63

Round Trip Time (RTT)

RTT (definition): time elapsed for a small packet to travel from Client to Server and back

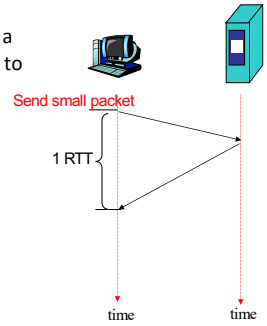
RTT is BOTH directions

- Distinct from one way delay

* Why isn't line horizontal?

Response time:

- 1 RTT



66

Opening a TCP Connection

3-way handshake

Set bits in TCP header

- SYN
- SYN-ACK
- ACK (can include data)

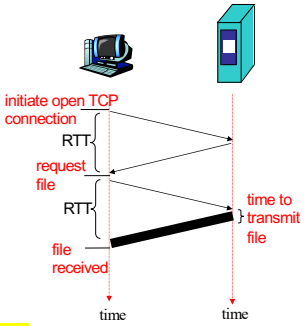
"Piggy-back" ACK:

3rd step ACK can include data

Response time:

1 RTT to open TCP connection

2 RTT to receive requested file (ignoring transmission delay)



69

HTTP TCP connections

Non-persistent:

- Key Feature:
- OS overhead for *each* new TCP connection
  - Buffers, timers, state variables

HTTP Header support:

Connection:

- Keep-alive
- close

Persistent:

- Key Feature:
- Existing connection used for all subsequent HTTP messages
- When to close?
  - Server reclaims resources

Parallel:

- Browsers often open parallel TCP connections to fetch referenced objects
- How to detect?

72

Non-Persistent HTTP

User enters URL: http://www.someSchool.edu/someDepartment/home.index

(Base HTML page with references to 10 embedded jpeg images)

1a. HTTP client initiates TCP connection to web server on port 80 www.someSchool.edu

1b. Web server at www.someSchool.edu is listening on port 80. "accepts" connection, notifies client

2. HTTP client sends HTTP GET *Request message* into TCP **socket.** Message indicates that client wants the object *someDepartment/home.index*

3. Web server receives Request message, forms *Response message* and sends requested object into socket

time

74

Non-Persistent HTTP (cont.)

4. HTTP server closes TCP connection.

5. HTTP client receives Response message (base HTML file). Parsing HMTL file, 10 referenced jpeg objects are found

time

Steps 1-5 repeated to fetch **each** of the 10 jpeg objects

Note to self: this sounds time consuming! --optimizations possible

75

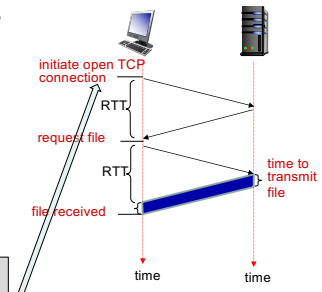
Non-persistent HTTP: response time

Receive Base HTML page:

1RTT+1RTT + file transmission time

Each additional object(s):

- Open TCP connection
- Send HTTP request and receive HTTP response
  - Consider object/file transmission time



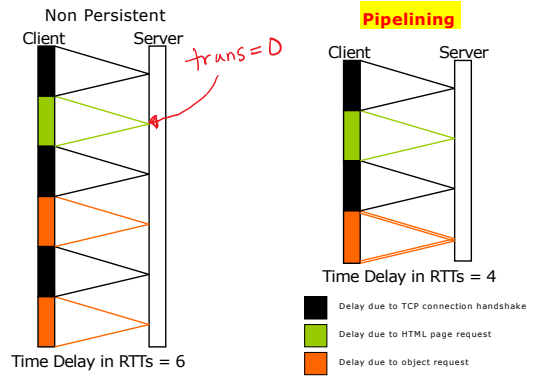
Total time to load entire web page (including objects) =
Base page: 2RTT
+
Each object: 2RTT + transmission time
Pipelining can reduce time

Where is the piggy-back ACK?

76

Non-Persistent TCP Connection

A web page with 2 objects:



77

Persistent HTTP

- Server leaves TCP connection open
  - Subsequent HTTP messages for objects continue to use existing connection
  - e.g., Additional referenced objects
- HTTP header: Connection: keep-alive

78

Persistent HTTP

User enters URL: http://www.someSchool.edu/someDepartment/home.index

(This example returns base HTML page with references to 10 embedded jpeg images)

1a. HTTP client initiates TCP connection to web server on port 80 www.someSchool.edu

1b. Web server at host www.someSchool.edu is listening on port 80. "accepts" connection, notifies client

2. HTTP client sends HTTP GET *Request message* into TCP **socket.** Message indicates that client wants the object *someDepartment/home.index*

3. Web server receives Request message, forms *Response message* containing requested object, and sends message into its socket

time

Nothing new on this page…

79

79

Persistent HTTP (cont.)

4. ~~HTTP server closes TCP connection.~~ STAYS OPEN

5. HTTP client receives Response message (base HTML file). Parsing HMTL file, 10 referenced jpeg objects are found

time

Steps 2,3,5 repeated for **each** of 10 jpeg objects
--possibly use "pipelining"

NOTICE STEP 1 and 4 NOT REPEATED

80

80

Persistent HTTP: Response time

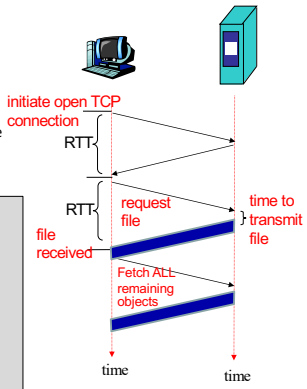
Base Page:

2RTT+ file transmission time

Additional embedded Object(s):

TCP Connection stays open!

1RTT to **grab them all** (pipelining) or one by one (serial). (Sometimes assume object/file transmission time = 0)



Total time to load entire web page (including objects using pipelining)

= Base page: 2RTT
+ 1RTT (receive all additional objects back-to-back) + file transmission times

What if not pipelining?

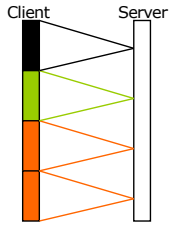
Where is the piggyback ACK?

81

Persistent TCP Connection

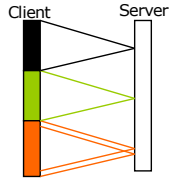
A web page with 2 objects:

Persistent without pipelining



Time Delay in RTTs = 4

Persistent with pipelining



Time Delay in RTTs = 3

Delay due to TCP connection handshake

Delay due to HTML page request

Delay due to object request

82

End of Ch2 part 1 slides

83